도착정보 산출 및 표출 흐름

목적과 범위

이 문서는 SmartBIS가 TAGO 차량 위치를 수집하고 다음 정류소·도착예정시간(ETA)을 산출해 Android 운영 화면에 표시하는 코드 흐름을 설명합니다. 내용은 작업공간 소스 기준이며, 실행 중 DB의 최신 차량 위치나 실시간 TAGO 응답을 확인한 결과와는 구분합니다.

전체 흐름

```
TAGO 차량 위치 API
  → TagoVehicleLocationService
  → bus_positions
  → GET /v1/arrival?vehicleNumber=...
  → ArrivalController
  → ArrivalService
      ├ 최근 위치 선택(2분 이내)
      ├ 노선·방향·정류장 순번으로 다음 정류소 계산
      ├ 거리와 100m 접근 여부 계산
      └ TagoArrivalEtaService에서 ETA 조회
  → ArrivalPrediction JSON
  → SmartBisRepository.currentArrival()
  → DisplayController (10초 간격)
      ├ 상단 정보 갱신
      ├ 다음 정류소 100m 이내면 전체화면 안내
      └ 응답 누락 시 상단 정보 삭제, 30초 후 전체화면 종료
  → NaturalDisplayView
```

Android 클라이언트

`client-android/app/src/main/java/com/smartbis/client/MainActivity.kt`

앱 구성요소를 초기화하고 서버 주소 및 현재 차량번호를 Repository/Controller에 연결합니다. 현재 차량번호가 API 조회 대상입니다.

`client-android/app/src/main/java/com/smartbis/client/model/SmartBisRepository.kt`

`currentArrival()`이 차량번호를 URL 인코딩하여 `GET /v1/arrival?vehicleNumber=...`를 호출하고 정상 JSON을 `ArrivalSnapshot`으로 변환합니다. HTTP 404는 도착정보 없음(`null`)으로 돌려줍니다.

`client-android/app/src/main/java/com/smartbis/client/controller/DisplayController.kt`

갱신 및 화면 전환을 관리합니다. 도착정보를 10초마다 조회하고, 응답이 없으면 상단 문구를 지웁니다. `within100m=true`면 전체화면 안내를 시작하고, 안내 중 응답이 30초 이상 없으면 추적을 초기화하고 일반 콘텐츠로

돌아갑니다.

`client-android/app/src/main/java/com/smartbis/client/model/ArrivalDisplayWindow.kt`

전체화면 안내 대상 정류소 하나를 추적합니다. 다음 정류소 100m 이내에서 목표 정류소를 기억하고, 차량이 그 정류소를 지난 뒤 현재 정류소에서 100m 이상 이동하면 안내를 종료합니다.

`client-android/app/src/main/java/com/smartbis/client/view/NaturalDisplayView.kt`

실제 UI 표시를 담당합니다. 상단 프레임에는 다음 정류소명과 ETA를, 전체화면에는 현재/다음 정류소·거리·도착/통과 상태를 표시합니다. ETA가 없으면 도착예정 확인 중, 응답이 없으면 현재 도착정보 없음으로 표시합니다.

SmartBIS 백엔드 API

`server/egov-backend/src/main/java/com/smartbis/backend/arrival/ArrivalController.java`

HTTP 진입점입니다. `GET /v1/arrival`의 필수 쿼리 `vehicleNumber`를 받아 계산 서비스에 전달하며, 산출 로직은 두지 않습니다.

`server/egov-backend/src/main/java/com/smartbis/backend/arrival/ArrivalService.java`

도착정보 계산의 핵심입니다.

1. `bus_positions`에서 요청 차량의 최신 위치 중 방향이 UP/DOWN이고 현재 시각 기준 2분 이내인 행을 선택합니다.
2. 위치 행의 `route_id`, `direction`, `node_id`, `node_order`로 `bus_route_stops`를 조회합니다.
3. 같은 노선·방향에서 현재 `node_order + 1`인 정류소를 다음 정류소로 정합니다.
4. 차량 좌표에서 다음 정류소까지 거리와 현재 정류소까지 거리를 계산하고, 다음 정류소 100m 이내인지 판정합니다.
5. 노선 ID와 다음 정류소 ID로 ETA를 요청합니다.
6. 결과 일부를 `arrival_predictions`에 저장하고 응답 모델을 반환합니다.

2분 이내 위치가 없거나 다음 정류소 행을 찾을 수 없으면 현재 코드는 404를 반환합니다.

`server/egov-backend/src/main/java/com/smartbis/backend/arrival/ArrivalPrediction.java`

API 응답 모델입니다. 차량·노선, 현재/다음 정류소 이름과 ID, 거리, 100m 접근 여부, ETA를 전달합니다.

TAGO 위치 및 도착예정 수집

`server/egov-backend/src/main/java/com/smartbis/backend/tago/TagoVehicleLocationService.java`

TAGO `getRouteAcctoBusLcList`를 호출해 차량번호, 노선, 정류장 ID/순번, 좌표를 수집하고 `bus_positions`에 저장합니다. 현재 소스는 활성 배정 차량과 연결된 활성 노선 ID를 중복 제거한 뒤, 실행마다 노선 하나를 순환 조회합니다. 조회 실패에는 백오프가 적용됩니다(일반 실패 2분, HTTP 429는 15분). 따라서 배정 DB에 남은 차량·노선과 수집 실패가 최신 위치의 신선도에 영향을 줍니다.

`server/egov-backend/src/main/java/com/smartbis/backend/tago/TagoArrivalEtaService.java`

TAGO `getSttnAcctoSpcifyRouteBusArvlPrearngeInfoList`를 호출해 도착예정 초를 분 단위로 올림합니다. 결과는 차량별이 아니라 `route_id + 다음 정류소 ID`로 캐시합니다. 현재 10초 간격으로 대상 하나를 순환 조회하며, 캐시 TTL은 대상 수에 따라 계산됩니다(최소 60초, 또는 대상 수 × 20초 + 20초). 따라서 ETA는 차량별 고유 예측값이 아니며 캐시 갱신 주기의 영향을 받습니다.

`server/egov-backend/src/main/java/com/smartbis/backend/tago/TagoApiRequestThrottle.java`

차량 위치·도착예정 조회가 TAGO 요청 간격 제한을 공유하도록 API 호출을 제한합니다.

증상별 확인 위치

증상	우선 대조할 값
다음 정류소명이 현재 위치와 다름	차량번호, 위치 행의 노선/방향/정류장 ID/순번, `bus_route_stops` 매핑
ETA가 낡거나 예상과 다름	TAGO 도착예정 응답, 노선·정류소 캐시 키, 갱신 주기와 TTL
도착정보가 사라짐	위치가 2분보다 오래됐는지, 다음 정류소 계산이 404인지, 수집 실패/백오프 여부
전체화면 안내가 안 나옴	`within100m` 값과 `ArrivalDisplayWindow` 상태
전체화면 안내가 중간에 끝남	API 응답 누락이 30초 이상인지

소스 분석과 운영 데이터의 구분

이 문서는 코드에 구현된 흐름과 정책을 설명합니다. 특정 차량의 최신 DB 위치, 실제 TAGO 응답, 단말이 받은 API 응답을 확인한 결과가 아닙니다. 특정 차량을 진단할 때는 같은 시각의 `bus_positions` 최신 행, 해당 노선/방향의 현재·다음 정류소 행, `/v1/arrival` 실제 응답, TAGO 로그, 단말 요청 차량번호를 함께 대조해야 합니다.